SWU Erasmus Staff Mobility Portal

Technical Document

Technical guide for local engineering use

Document	Technical system and deployment guide
Audience	Local engineers, technical reviewers, and maintainers
Scope	Current repository state and current deployment path
Version	March 2026

Contents

Purpose of this document

This document explains the Erasmus staff mobility portal from an engineering point of view. It is meant for a local engineer who needs to understand what the website does, how the code is organised, how the main workflows work, and how to run or deploy the system on another machine inside the university.

The document is based on the current repository and describes the system as it exists now. Later hosting work, broader integrations, and future workflow extensions are separate work.

System summary

The portal is an internal web application for Erasmus staff mobility administration. It supports three user roles:

- staff users, who maintain their own profile, create and submit mobility cases, upload required documents, and follow review results
- officers, who review submitted cases, add comments, review documents, change case status, archive completed cases, and use reports
- admins, who manage users, approvals, master data, settings, and the audit log

The application runs as a Next.js application with a PostgreSQL database and a private local file store for uploaded documents. Authentication is credentials-based. Data access is handled through Prisma. Automated tests cover the main application layers and the critical cross-role workflow.

Technology stack

Area	Current choice
Frontend framework	Next.js App Router
Language	TypeScript
UI layer	React, Tailwind CSS, local UI components under `src/components/ui`
Forms	React Hook Form with Zod validation
Authentication	Auth.js with credentials provider and Prisma adapter
Database	PostgreSQL
ORM	Prisma
Document storage	Local filesystem through a storage abstraction
Unit and integration tests	Vitest and React Testing Library
Browser tests	Playwright

The main package and script definitions are in `package.json`. Prisma configuration is in `prisma.config.ts`. Local database startup uses `docker-compose.yml`.

Repository layout

The repository is split into application code, schema and seed files, tests, and operational documentation.

Main folders

Path	Purpose
`src/app`	Next.js pages, layouts, and API routes
`src/components`	UI building blocks and feature-specific components
`src/lib`	Application services, validation, auth helpers, reporting, review logic, storage, and shared utilities
`prisma`	Prisma schema, migrations, and seed script
`tests`	Unit, integration, component, and end-to-end tests
`docs`	Startup, testing, storage, deployment, progress, and user-facing documentation

Important application subfolders

Path	Purpose
`src/components/auth`	Login, registration, and sign-out UI
`src/components/cases`	Case forms, case tables, status badges, and document panels
`src/components/review`	Officer review filters, review forms, and review tables
`src/components/reports`	Reporting filters, summary tables, export actions, and case lists
`src/lib/auth`	Authentication, approval, guards, and auth navigation logic
`src/lib/profile`	Staff profile read and update logic
`src/lib/mobility-case`	Staff case creation, editing, submission, and detail logic
`src/lib/documents`	Upload policy, version handling, and download authorization
`src/lib/review-workflow`	Officer case list, case detail, comments, document review, and status changes
`src/lib/reporting`	Server-side report aggregation and CSV export generation
`src/lib/storage`	Storage abstraction and local file driver
`src/lib/validation`	Zod schemas and validation helpers

Application layout and navigation

Public area

The public area is defined by the root layout in `src/app/layout.tsx`. It includes the top bar, public navigation, language toggle, and footer. The public navigation currently exposes:

- `/`
- `/status`
- `/login`
- `/register`

When a user is already signed in, the public layout also exposes `/dashboard`.

Protected workspace

The protected workspace is defined in `src/app/dashboard/layout.tsx`. That layout does four things:

- requires approved authentication before rendering the dashboard surface
- shows a role-based left navigation
- shows signed-in account context, role, and approval state
- provides a shared header with language toggle and sign-out

Navigation is generated in `src/lib/navigation.ts`. The role filter is explicit. Staff, officer, and admin users do not see the same pages.

Main page routes

Route	Access	Purpose
`/`	Public	Home page and entry point
`/login`	Public	Credentials login
`/register`	Public	Staff self-registration
`/pending-approval`	Public	Pending-account holding page
`/status`	Public	Local status page
`/dashboard`	Approved users	Common protected landing page
`/dashboard/profile`	Approved users	Personal profile editing
`/dashboard/staff`	Staff	Staff dashboard
`/dashboard/staff/cases/new`	Staff	New case form
`/dashboard/staff/cases/[caseId]`	Staff owner only	Staff case detail page
`/dashboard/officer`	Officer, Admin	Officer dashboard
`/dashboard/officer/cases`	Officer, Admin	Review register
`/dashboard/officer/cases/[caseId]`	Officer, Admin	Review detail page
`/dashboard/reports`	Officer, Admin	Reporting and CSV exports
`/dashboard/admin`	Admin	Admin dashboard
`/dashboard/admin/users`	Admin	User lifecycle management

`/dashboard/admin/master-data`	Admin	Master data and settings
`/dashboard/admin/audit-log`	Admin	Audit log viewer

API route groups

Route group	Purpose
`/api/auth/[...nextauth]`	Auth.js handlers
`/api/register`	Staff registration
`/api/profile`	Profile updates
`/api/staff/cases`	Staff case creation and list actions
`/api/staff/cases/[caseId]`	Staff case update actions
`/api/staff/cases/[caseId]/documents`	Staff document uploads
`/api/review/cases/[caseId]/comments`	Officer comments
`/api/review/cases/[caseId]/documents/review`	Officer document review actions
`/api/review/cases/[caseId]/missing-documents`	Officer missing-document actions
`/api/review/cases/[caseId]/status`	Officer case status transitions
`/api/admin/users/[userId]`	Admin approval, role, and account actions
`/api/admin/master-data`	Admin master-data and settings updates
`/api/reports/export/...`	CSV export endpoints
`/api/documents/versions/[versionId]/download`	Private document download route
`/api/health`	Technical health checks
`/api/locale`	Language cookie updates

User roles and access model

Role model

User roles are defined in the Prisma schema as `STAFF`, `OFFICER`, and `ADMIN`. User approval state is separate and includes `PENDING`, `APPROVED`, `REJECTED`, and `DEACTIVATED`.

This separation is important because role alone is not enough. A user can have the `STAFF` role and still be unable to use the protected area until the account is approved.

Authentication flow

Auth.js is configured in `src/auth.ts`. The credentials provider validates incoming credentials, delegates the actual credential check to `src/lib/auth/service.ts`, and then enriches the session token with role and approval state.

The authentication service performs:

- email normalisation
- password hash comparison with `bcryptjs`
- approval-state checks
- explicit return codes for invalid credentials, pending approval, rejected account, and deactivated account

Authorisation model

Protected actions are not left to the frontend. The current system uses:

- dashboard-level guards
- role-based navigation
- server-side checks in protected API routes
- ownership checks for staff-owned cases and documents

This means that staff users cannot access another staff user's case or private file just by changing a URL.

Main functional areas

Registration and account approval

Staff registration is handled through `/register` and `/api/register`. The service in `src/lib/auth/service.ts` creates a pending staff account and records an audit entry.

Admin approval and rejection are handled from `/dashboard/admin/users`. The same service layer updates approval state and writes audit records for those decisions.

Profile management

Profile handling is implemented in `src/lib/profile/service.ts`. The profile model links the user to:

- academic title
- faculty
- department

The profile service loads valid selections, preserves legacy values safely if needed, and validates faculty and department consistency during updates.

Mobility case management

Staff case logic is implemented in `src/lib/mobility-case/service.ts`. A case can be:

- created as a draft
- reopened and edited later
- submitted once required fields are complete

The service writes explicit status-history records. Draft and submit flows are intentionally separate. A draft can be incomplete. Submission requires a valid academic year, mobility type, and date range.

Document handling

Document handling is implemented in `src/lib/documents/service.ts` and `src/lib/validation/documents.ts`.

The current business document types are:

- Mobility Agreement
- Final Certificate of Attendance

Each upload is stored as a new version. The system preserves old versions, stores original filename and metadata, and tracks which version is current.

Officer review workflow

Officer review logic is implemented in `src/lib/review-workflow/service.ts`. Officers can:

- view all cases
- search and combine filters
- open case detail pages
- add comments
- review current document versions
- mark missing documents
- change case status
- archive completed cases

Document review state and case workflow state remain separate on purpose. Rejecting a document does not automatically change the whole case status.

Reporting and CSV export

Reporting is handled in `src/lib/reporting/service.ts`, `src/lib/reporting/csv.ts`, and the export routes under `src/app/api/reports/export`.

Current reporting supports:

- mobility count by academic year
- by faculty
- by department
- by mobility type
- by host country
- by host institution
- by status
- open versus completed
- cases without mobility agreement
- cases without final certificate

Audit logging

Important protected actions are stored explicitly in the `AuditLog` model. This includes user lifecycle actions, case creation and submission, document events, review actions, status changes, and settings changes.

Data model overview

Core entities

The main schema lives in `prisma/schema.prisma`. The most important models are:

- `User`
- `Faculty`
- `Department`
- `AcademicYear`
- `CaseStatusDefinition`
- `SelectOption`
- `MobilityCase`
- `MobilityCaseComment`
- `MobilityCaseStatusHistory`
- `MobilityCaseDocument`

- `MobilityCaseDocumentVersion`
- `AuditLog`
- `UploadSetting`
- `ReportSetting`

Important design choices

- User role and approval state are separate.
- Faculty and department are relational, not free-text.
- Case status history is stored explicitly.
- Document review state is separate from case status.
- Document versions are stored explicitly with a current-version pointer.
- Audit records are explicit rows, not inferred only from timestamps.

Document storage and private downloads

The storage abstraction is in `src/lib/storage/driver.ts`. The current concrete implementation is `src/lib/storage/local-driver.ts`.

The local driver:

- writes files under the configured private storage root
- blocks path traversal by resolving and checking the target path
- keeps files outside the public web root

The download route `/api/documents/versions/[versionId]/download` performs permission checks before reading a file. That means a document is never exposed as a direct public static URL.

Validation and security model

Validation

The repository uses Zod schemas under `src/lib/validation`. Validation exists for:

- login and registration
- profile updates
- master data
- mobility cases
- review actions
- document uploads

Current security controls

Current security-related controls include:

- server-side role and approval checks
- ownership checks for staff case and document access
- password hashing
- private document storage
- guarded download routes
- upload validation for extension, size, declared MIME type, and basic file signatures
- blocking of risky PDF active-content markers
- blocking of unsafe DOCX markers such as macro and embedded-object indicators
- CSV formula neutralisation for exports
- audit logging of protected actions

Current security limits

The current repository does not yet include:

- full antivirus or sandbox-based malware scanning
- password reset or email verification
- infrastructure monitoring and alerting
- multi-instance shared storage

Testing and quality

Testing strategy is documented in `docs/testing-strategy.md`. The repository currently includes:

- unit tests for pure validation and helper logic
- integration tests for services and form behaviour
- component tests for visible UI behaviour
- Playwright browser tests for the main role-based workflows

The most important browser reference is `tests/e2e/critical-portal-journey.spec.ts`. That file exercises the main cross-role workflow from registration through archive and reporting.

Current quality commands:

```
npm run lint
npm run typecheck
npm run test
npm run test:e2e
npm run build
```

Local installation on another machine

Repository source

The current source repository is:

`https://github.com/Nico-Ab/Erasmus-Website.git`

For a fresh machine, the normal starting point is:

```
git clone https://github.com/Nico-Ab/Erasmus-Website.git
cd Erasmus-Website
```

Prerequisites

The current local setup expects:

- Node.js 24 or newer
- npm
- Docker Desktop or another Docker runtime
- a machine that can expose PostgreSQL on port 5432

Configuration files

Relevant startup files:

- `.env.example`
- `docker-compose.yml`
- `prisma.config.ts`
- `docs/local-startup.md`

First-time local startup

For a fresh local machine:

```
docker compose up -d
npm install
npm run db:migrate
npm run seed
npm run dev
```

Then open:

`http://127.0.0.1:3000`

Useful local checks:

```
http://127.0.0.1:3000/status
http://127.0.0.1:3000/api/health
```

Important environment values

Variable	Purpose
`APP_URL`	Base app URL
`DATABASE_URL`	PostgreSQL connection string
`AUTH_SECRET`	Auth.js secret, must be strong in real use
`AUTH_TRUST_HOST`	Host trust setting for Auth.js
`STORAGE_DRIVER`	Current storage backend, local by default
`STORAGE_LOCAL_ROOT`	Private storage root for uploaded files
`MAX_UPLOAD_SIZE_MB`	Upload size limit
`ALLOWED_UPLOAD_EXTENSIONS`	Allowed upload extensions
`DEFAULT_LOCALE`	Default interface language

Seed data

The seed script is in `prisma/seed.ts`. It upserts:

- demo admin, officer, and staff accounts
- pending, rejected, and deactivated account examples
- faculties, departments, academic years, statuses, and select options
- upload and report settings
- sample draft and submitted cases

For technical review and testing, running `npmrunseed` is useful. For a real internal rollout with real data, it should be used carefully.

Deployment process on another internal system

What the repository already supports

The repository is already structured for a production-like single-host deployment. It has:

- environment-based configuration
- committed Prisma migrations
- a deployment-style migration command
- a build command
- a production start command
- server-side protected actions

Current recommended deployment model

The current supported deployment model is a single internal machine with:

- PostgreSQL
- the Next.js app process
- private document storage on local disk

This is the same model already used locally, just run in production mode instead of development mode.

Target deployment profile

At the moment, the cleanest supported deployment is a single-machine internal installation. In practice that means:

- one application host
- one PostgreSQL database, either local through Docker or provided separately
- one private storage directory on the same host for uploaded files

This fits the current codebase because document storage is local filesystem based, the application is started as one Next.js process, and the main operational docs are written around that model.

Important deployment notes before starting

There are two practical details in the current repository that an engineer should know before deployment.

First, the checked-in Docker setup is fine for local development, but it uses an obvious default database password in `docker-compose.yml`. That file should not be copied into an internal environment unchanged.

Second, the current `npmrunstart` script binds the Next.js server to `127.0.0.1:3000`. That is correct for local use and for temporary tunnel-based sharing from the same machine. For a server that must be reachable by other machines on an internal network, the final hosting layer needs to account for that. In practice this usually means either adjusting the start binding for the deployment target or placing the app behind a reverse proxy on the same host. The repository does not yet contain that outer hosting configuration.

Deployment sequence

For a clean deployment on another internal machine:

1. Install Node.js and Docker on the target machine.
2. Clone the repository from GitHub.
3. Create a real `.env` from `.env.example`.
4. Set a strong `AUTH_SECRET`.

5. Set `DATABASE_URL` for the target PostgreSQL instance.
6. Set `APP_URL` to the URL that users should open on that machine or host.
7. Set `STORAGE_LOCAL_ROOT` to a private directory on disk that will be kept between restarts and backups.
8. Prepare the database.
9. Install dependencies with `npminstall`.
10. Apply committed migrations with `npmrundb:migrate:deploy`.
11. If the target is a demo or evaluation system, optionally run `npmrunseed`.
12. Run `npmrunbuild`.
13. Start the application with `npmrunstart`.

Database preparation

The repository supports two realistic ways to prepare PostgreSQL on the target machine.

The first option is to reuse the local Docker model from `docker-compose.yml`. That is the fastest way to get a working internal instance. If that path is chosen, the default database name, user, and password should be reviewed and changed before the system is treated as a real shared installation.

The second option is to point `DATABASE_URL` at an existing PostgreSQL instance that the university already maintains. From the application side, that is enough as long as the target database is reachable and the Prisma migration command can run against it.

Storage preparation

Document storage is not optional in this system. Staff users upload document versions, and those files are stored outside the public web root. Before deployment, the engineer should decide where the private storage directory will live.

The directory configured through `STORAGE_LOCAL_ROOT` should:

- be writable by the application process
- not sit under a public static web directory
- remain available after application restarts
- be included in backup planning together with the database

If that directory is lost, document metadata may still exist in the database while the underlying files are gone. For that reason, database backup alone is not enough.

Production-like commands

```
git clone https://github.com/Nico-Ab/Erasmus-Website.git
cd Erasmus-Website
copy .env.example .env
docker compose up -d
npm install
```

```
npm run db:migrate:deploy
npm run build
npm run start
```

What to edit in the environment file

The minimum changes before a real internal handoff are:

- replace `AUTH_SECRET`
- confirm `APP_URL`
- confirm `DATABASE_URL`
- confirm `STORAGE_LOCAL_ROOT`
- review upload settings such as `MAX_UPLOAD_SIZE_MB` and `ALLOWED_UPLOAD_EXTENSIONS`

If the deployment is intended for Bulgarian-first use, the engineer can also change `DEFAULT_LOCALE` from `en` to `bg`.

Optional demo setup

If the engineering team wants a seeded demonstration instance first:

```
npm run seed
```

That is suitable for local testing or a demo server. It should not be confused with a clean production rollout.

First checks after deployment

After the application has started, the first checks should be simple:

1. open the configured base URL
2. open `/status`
3. open `/api/health`
4. sign in with a seeded or known account
5. open a protected dashboard page
6. if the system will be used for real workflows, upload and download one document through the protected flow

This is enough to confirm that the application server, database, auth layer, and private storage are all working together.

Routine update process

For later updates on the same host, the basic sequence stays simple:

1. pull the latest repository state
2. review any environment changes
3. run `npminstall` if dependencies changed
4. run `npmrundb:migrate:deploy`
5. run `npmrunbuild`
6. restart the production application process

If the update includes schema changes, the migration step should not be skipped.

What the local engineer will still need to decide

The repository brings the application itself, but an internal deployment still needs a few local decisions:

- whether PostgreSQL is run through the included Docker setup or an existing university database
- where the private storage directory will live
- how the application process is kept running after restarts
- whether the application is only for same-machine use, internal network use, or temporary public sharing
- whether a reverse proxy and TLS are required for the target environment

What is not yet included in the repository

The repository does not currently include:

- reverse proxy configuration
- TLS certificate management
- service manager files such as systemd units
- backup scheduling
- central log shipping
- monitoring and alerting setup

An internal engineer can still deploy the application on a single machine, but those operational layers need to be added outside the application repository.

Operational notes for maintainers

Database

Database schema is managed through Prisma migrations. For future changes, the repository already supports:

- development migrations with `npmrundb:migrate`
- deployment migrations with `npmrundb:migrate:deploy`
- schema push with `npmrundb:push`

Storage

Uploaded files are stored privately on local disk. The current implementation is suitable for:

- local development
- local evaluation
- single-host internal deployment

For future multi-host deployment, storage should move to a shared persistent backend. The abstraction is already in place for that change.

External temporary access

If temporary public access is needed without paid hosting, the repository already documents tunnel-based sharing in `docs/external-sharing.md`. That is suitable for review sessions, not for permanent hosting.

Current limitations and scope boundaries

The repository is still limited in a few areas. Current boundaries include:

- no EWP integration
- no public Erasmus portal
- no digital-signature workflow
- no password reset or account recovery
- no bulk officer actions
- no saved views or pagination for large queues
- no non-CSV export formats
- no full malware scanning

Conclusion

The current repository is a working local v1 system for an internal Erasmus staff mobility portal. It contains the application structure, protected role-based workflows, data model, validation, private document handling, reporting, audit logging, and a deployment path for another internal machine.

For a local engineer, the important conclusion is simple: the system is ready to run on another machine with Node.js, PostgreSQL, Docker, environment configuration, Prisma migrations, and a private storage directory. The current codebase is also organised clearly enough to be maintained and extended without needing a full redesign first.